

静かに見守る、IoTプロダクト

植物見守り

植物環境モニタリングシステム

目次

- 1.制作背景・課題
- 2.コンセプト
- 3.システム構成
- 4.デバイス構成
- 5.アプリ画面
- 6.実演
- 7.今後の展望



制作背景・課題

植物の育成には、初心者にとって
難しい課題があります。
これらの解決を目指しました。



状態が分かりにくい

見た目だけでは元気かどうか判断しづらい



水やりの判断が難しい

タイミングを逃しやすい



過去の変化が分からない

環境データの記録が手元にない



初心者は育成に失敗しやすい

気づいた時には手遅れ、ということも

「育てたいけれど、ちゃんと管理できるか不安」な人へ

メインユーザー

観葉植物を 育て始めた初心者

抱えやすい不安

- 水やりのタイミングが分からない
- 植物が元気なのか判断しにくい
- 外出中の状態が分からない
- 過去の環境変化が分からない

「枯れてから気づく」を減らす

植物見守りが提供する体験

見る

温度・湿度・土壌水分などを
データとして見える化

分かる

アイコンと短いメッセージで
状態をひと目で把握

気づける

状態が悪化した時だけ
そっと通知

振り返る

過去の環境変化を
グラフで確認

「分からない」を「分かる」に変え、管理の負担を減らす

「静かに見守る」

普段は何もしなくていい。必要な時だけ、そっと知らせる。



状態をひと目で

アイコンと短いメッセージで
直感的に把握



必要な時だけ通知

状態が悪化した時だけ
そっと知らせる



今日のまとめで把握

平均の環境データを
コンパクトに表示

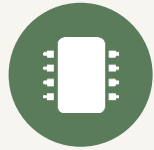
システム構成

センサーからアプリまで、
Wi-Fi経由でひとつながりに
データが流れる構成です。



センサー

温湿度・土壌水分・照度



ESP32

ATOM Matrix



Node.js

APIサーバー



SQLite

データ保存



Flutter

スマホアプリ



デバイス構成

部品	説明
ATOM Matrix	Wi-Fi/Bluetooth対応マイコンモジュール
ATOM PortABC拡張ベース	拡張機
ENV Ⅲ	温度・湿度・気圧センサー(I2C接続)
土壌水分センサー	アナログ出力(GPIO接続)
光センサーユニット	照度デジタル変換(I2C接続)
小型モバイルバッテリー	USB-C出力

- ATOM Matrixを中心に、ENV Ⅲ(温湿度・気圧)センサー、土壌センサー水分、照度センサーを搭載
- モバイルバッテリーでの駆動に対応
- ステータスLEDにより通信・エラー状態を色で表示

アプリ画面①ーホーム・植物情報



ホーム:4項目の環境データを一目で

温度・湿度・土壌水分・照度をリアルタイム表示。湿度/照度は日次評価バッジ、土壌水分は「水やりしました」ボタンと連携



植物情報:植物の基本情報とケア条件を確認

植物名・植物種の表示と変更

アプリ画面②ー履歴・通知・設定



履歴:全項目・全期間をグラフで振り返り

土壌水分/温度/湿度/照度を切り替え表示。24h/7d/30dの期間切り替え、水やり記録をグラフ上にマーカー表示



通知:悪化の瞬間と日次要ケアを通知

リアルタイム系(温度・土壌水分)と日次系(湿度・照度)の判定タイミングの違いをアプリ内でも明示



設定:通知条件をきめ細かく調整

カテゴリ別ON/OFF、デバイス接続状態をこの画面で管理

動作実績:ESP32からの自動送信



```
plant_sensor_integrated.ino secrets.h

出力 シリアルモニタ 115200 baud

14:32:01 Temp=24.8C Humidity=63.2% Soil=38.10 Illuminance=850lx
14:32:01 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
14:32:31 Temp=24.7C Humidity=63.0% Soil=38.05 Illuminance=842lx
14:32:31 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
14:33:01 Temp=24.9C Humidity=62.6% Soil=37.90 Illuminance=855lx
14:33:01 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
14:33:31 Temp=24.8C Humidity=62.9% Soil=37.85 Illuminance=849lx
14:33:31 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
14:34:01 Temp=25.0C Humidity=62.4% Soil=37.70 Illuminance=861lx
14:34:01 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
14:34:31 Temp=24.9C Humidity=62.7% Soil=37.65 Illuminance=858lx
14:34:31 POST /sensor -> HTTP 201 : {"plant_id":1,"status":"healthy"}
```



Wi-Fi経由で自動送信(30秒間隔)



4センサー全項目が実データで反映

温度・湿度・土壌水分・照度をENV III/BH1750/U019から取得



Flutter側も自動ポーリングで反映

サーバー受信 → 状態判定 → アプリ表示までEnd to Endで確認

デモの流れ

STEP 1

センサーデバイス紹介

実機のATOM Matrixと外観・部品構成

STEP 2

ホーム画面

今の温度・湿度・状態をリアルタイムに表示

STEP 3

状態悪化を再現

土壌水分を模擬的に下げ、通知が届く様子を実演

STEP 4

履歴画面

これまでの環境変化をグラフで振り返る

「監視する機器」から「管理を支えるサービス」へ

個人ユーザー

日常の管理負担を減らす

- 毎日アプリを開かなくてもよい
- 水やりの判断材料が得られる
- 異常時だけ通知を受け取れる
- 過去の変化を振り返れる

複数植物の管理

管理対象が増えても把握しやすい

- 植物ごとの状態を管理
- 複数デバイスへ拡張できる
- データを一元的に確認できる
- 管理規模に合わせて発展できる

サービスとしての展開

クラウド・通信機能で広げる

- Firebaseなどへのクラウド移行
- ログイン・セキュリティ強化
- Push通知・BLEへの対応
- 植物ごとの管理へ発展

個人の安心 → 管理の効率化 → サービス化

※市場規模・販売価格などの事業性検証は今後の課題。ここでは、製品化を見据えた価値と拡張可能性を示しています。

今後の展望



通知機能の強化

タイミングのカスタマイズ等



複数植物・複数デバイス対応

現状は1台1株の単独構成



履歴グラフ機能の改善

期間指定・比較表示など



クラウド対応

Node.js → Firebaseへの移行



セキュリティ関連の強化

認証機能の検討



筐体制作

3Dプリンターでの型番作成・配色

ご清聴ありがとうございました

植物見守り — 植物環境モニタリングシステム

プロジェクト

植物見守りプロジェクト

GitHub

× × ×

静かに見守る、IoTプロダクト

植物見守り

植物環境モニタリングシステム
（使いませんが残しておきます）

開発の進捗状況(使いませんが残しておきます)

完了

- ✓ Figmaデザイナー式(画面・デザインシステム・ユーザーフロー)
- ✓ Flutter UI試作(5画面)
- ✓ Node.js API(plants / sensor / history / notifications)
- ✓ アプリ ⇄ サーバーの実接続
- ✓ ESP32からのWi-Fi自動送信
- ✓ 状態悪化時の通知自動生成
- ✓ 履歴グラフの実装(24h / 7d / 30d)
- ✓ デバイスペアリンク機能の実装

今後の対応

- 新規登録画面を作成する(現状はターミナルで手動登録)
- Push通知を追加する
- Bluetooth Low Energyを実装する